



The University of Azad Jammu and Kashmir

Name: Zarnab Fatima

Roll NO: 2024-SE-14

Semester:4th

Course Title: Database System

Course Code: CS-2204

Submitted To: Engr. Awais Rathore

Department of Software Engineering

LAB 11

1. Introduction

Scalar functions operate on one row at a time and return one value for each row. In this lab, string functions are used to clean, format, extract, and modify text data stored in the Customer and Product tables.

The main string functions used in this lab are:

- TRIM()
- UPPER()
- LOWER()
- CHAR_LENGTH()
- CONCAT()
- SUBSTRING()
- LEFT()
- REPLACE()
- LPAD()
- LOCATE()

Create Database



2. Create and Insert Table (Customer , Product)

Table	Action	Rows	Type	Collation	Size	Overhead
☐ customer	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ product	★ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
2 tables	Sum	0	InnoDB	utf8mb4_general_ci	32.0 KiB	0 B

SELECT * FROM `customer`

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

				CustID	CustName	Email	City	Phone	JoinDate	DOB
☐				1	Ali Khan	ali.khan@MAIL.com	Lahore	0300-1112233	2022-01-15	1995-04-12
☐				2	Sara Iqbal	sara@example.com	Karachi	0301-4445566	2022-04-22	1998-11-20
☐				3	HAMZA RAZA	hamza@example.com	Lahore	0302-7778899	2023-02-10	1997-08-05
☐				4	Ayesha Noor	NULL	Islamabad	0303-1234567	2023-05-18	1999-02-14
☐				5	bilal ahmed	bilal@MAIL.COM	Karachi	0304-2345678	2023-09-01	2000-06-30
☐				6	Fatima Sheikh	fatima@example.com	NULL	0305-3456789	2024-01-12	1996-10-25
☐				7	Usman Tariq	usman@example.com	Lahore	NULL	2024-06-30	2001-03-18
☐				8	Maira Javed	maira@example.com	Islamabad	0307-5678901	2024-08-25	1994-12-09

☐ Check all

☐ All selected

Edit

Copy

Delete

Refresh

SELECT * FROM `product`

Profiling

Edit inline

Edit

Explain SQL

Create PHP code

Refresh

Show all

Number of rows: 25

Filter rows: Search this table

Sort by key: None

Extra options

				ProdID	ProdName	Category	Price	StockQty	LaunchDate
☐				101	Laptop Pro 15	Electronics	185000.00	12	2023-03-10
☐				102	Wireless Mouse	Electronics	2500.00	50	2022-07-22
☐				103	USB-C Cable	Electronics	800.00	100	2021-11-05
☐				104	Office Chair	Furniture	18500.00	8	2023-01-15
☐				105	Standing Desk	Furniture	45000.50	5	2024-02-28
☐				106	Notebook A4	Stationery	350.00	200	2020-04-01
☐				107	Ballpoint Pen 10pk	Stationery	450.00	150	2020-04-01
☐				108	Coffee Beans 1kg	Grocery	1899.99	30	2023-09-20
☐				109	Green Tea Box	Grocery	650.00	45	2022-12-12
☐				110	Bluetooth Speaker	Electronics	7500.00	18	2024-05-18

☐ Check all
 ☐ All selected
 Edit
 Copy
 Delete
 Refresh

3.PART B — NUMERIC AND DATE/TIME FUNCTIONS

In Part B, numeric and date/time scalar functions are used to perform calculations, rounding, date extraction, date formatting, date differences, and date arithmetic. These functions transform raw database values into meaningful and human-readable information.

B1 — Apply a 15% Discount

Objective:

Apply a 15% discount to every product and display the original price and discounted price rounded to two decimal places.

SQL Query:

```
SELECT
    ProdName,
    Price,
    ROUND(Price * 0.85, 2)
AS
DiscountedPrice
FROM Product;
```

Functions Used: ROUND()

Explanation: A 15% discount means the customer pays 85% of the original price. ROUND() is used to round the result to two decimal places. **Formula:** Discounted Price = Price × 0.85

`SELECT ProdName, Price, ROUND(Price * 0.85, 2) AS DiscountedPrice FROM Product;`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: Filter rows: Sort by key:

Extra options

				ProdName	Price	DiscountedPrice
☐				Laptop Pro 15	185000.00	157250.00
☐				Wireless Mouse	2500.00	2125.00
☐				USB-C Cable	800.00	680.00
☐				Office Chair	18500.00	15725.00
☐				Standing Desk	45000.50	38250.43
☐				Notebook A4	350.00	297.50
☐				Ballpoint Pen 10pk	450.00	382.50
☐				Coffee Beans 1kg	1899.99	1614.99
☐				Green Tea Box	650.00	552.50
☐				Bluetooth Speaker	7500.00	6375.00

Console

B2 —Calculate 17% Sales Tax

Objective:

Calculate 17% sales tax for every product and display the final price including tax.

SQL Query:

SELECT

ProdName,

ROUND(Price * 0.17, 2) AS

Tax,

ROUND(Price + (Price * 0.17), 2) AS PriceWithTax

FROM Product;

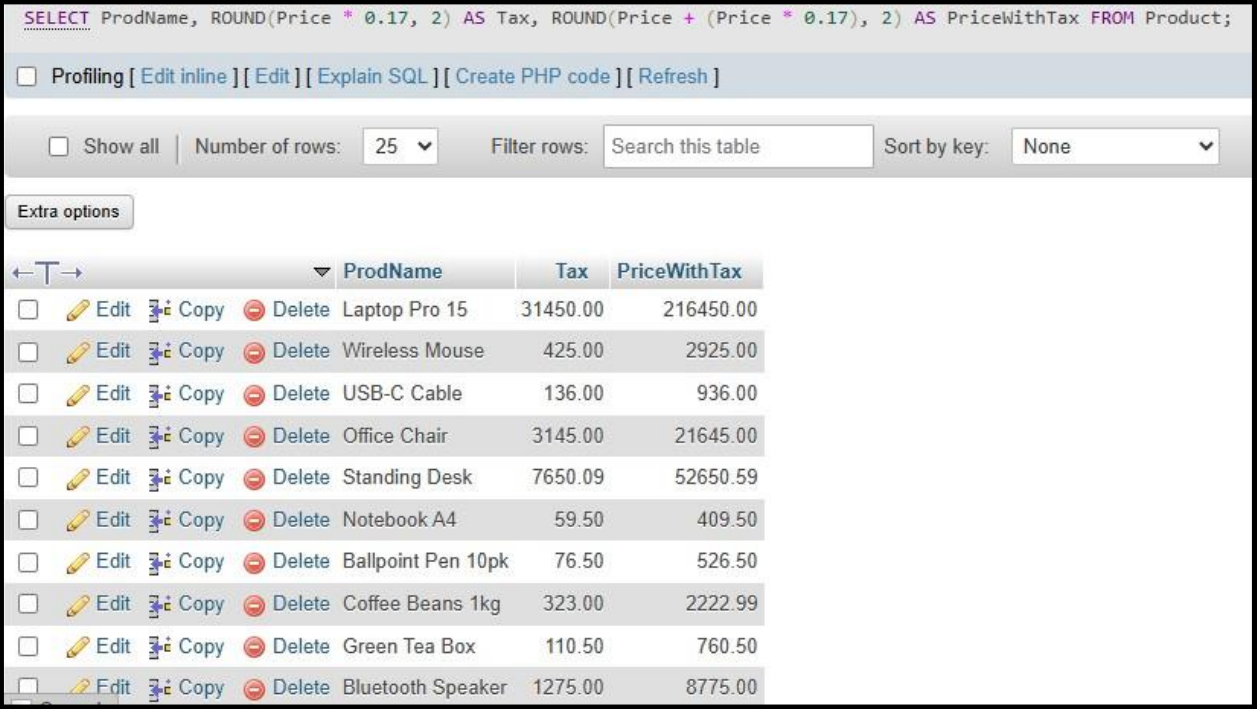
Functions Used: ROUND()

Explanation: The tax is calculated by multiplying the product price by 17%. The tax is then added to the original price.

Formula:

Tax = Price × 0.17

Price With Tax = Price + Tax



```
SELECT ProdName, ROUND(Price * 0.17, 2) AS Tax, ROUND(Price + (Price * 0.17), 2) AS PriceWithTax FROM Product;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				ProdName	Tax	PriceWithTax
☐				Laptop Pro 15	31450.00	216450.00
☐				Wireless Mouse	425.00	2925.00
☐				USB-C Cable	136.00	936.00
☐				Office Chair	3145.00	21645.00
☐				Standing Desk	7650.09	52650.59
☐				Notebook A4	59.50	409.50
☐				Ballpoint Pen 10pk	76.50	526.50
☐				Coffee Beans 1kg	323.00	2222.99
☐				Green Tea Box	110.50	760.50
☐				Bluetooth Speaker	1275.00	8775.00

B3 — Floor and Ceiling of Price Divided by 1000

Objective:

Calculate the floor and ceiling values of each product's price divided by 1000.

SQL Query:

SELECT

ProdName,

Price,

FLOOR(Price / 1000) AS

FloorVal,

CEIL(Price / 1000) AS

CeilVal

FROM Product;

Functions Used: FLOOR(), CEIL()

Explanation: FLOOR() returns the largest integer less than or equal to the value.

CEIL() returns the smallest integer greater than or equal to the value.

The screenshot shows a SQL query execution interface. At the top, the query is: `SELECT ProdName, Price, FLOOR(Price / 1000) AS FloorVal, CEIL(Price / 1000) AS CeilVal FROM Product;`. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A table view is selected, showing 25 rows. The table has columns: ProdName, Price, FloorVal, and CeilVal. The data is as follows:

	ProdName	Price	FloorVal	CeilVal
☐	Laptop Pro 15	185000.00	185	185
☐	Wireless Mouse	2500.00	2	3
☐	USB-C Cable	800.00	0	1
☐	Office Chair	18500.00	18	19
☐	Standing Desk	45000.50	45	46
☐	Notebook A4	350.00	0	1
☐	Ballpoint Pen 10pk	450.00	0	1
☐	Coffee Beans 1kg	1899.99	1	2
☐	Green Tea Box	650.00	0	1
☐	Bluetooth Speaker	7500.00	7	8

B4 — Round Price to the Nearest Hundred

Objective:

Round each product's price to the nearest hundred.

SQL Query:

SELECT

ProdName,

Price,

ROUND(Price, -2) AS

RoundedPrice

FROM Product;

Function Used: ROUND()

Explanation: The second argument -2 tells MySQL to round the number to the nearest hundred.

Example:

185000 → 185000

18550 → 18600

✓ Showing rows 0 - 9 (10 total, Query took 0.0005 seconds.)

```
SELECT ProdName, Price, ROUND(Price, -2) AS RoundedPrice FROM Product;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: | Filter rows: | Sort by ke

Extra options

				ProdName	Price	RoundedPrice			
☐		Edit		Copy		Delete	Laptop Pro 15	185000.00	185000
☐		Edit		Copy		Delete	Wireless Mouse	2500.00	2500
☐		Edit		Copy		Delete	USB-C Cable	800.00	800
☐		Edit		Copy		Delete	Office Chair	18500.00	18500
☐		Edit		Copy		Delete	Standing Desk	45000.50	45000
☐		Edit		Copy		Delete	Notebook A4	350.00	400
☐		Edit		Copy		Delete	Ballpoint Pen 10pk	450.00	500
☐		Edit		Copy		Delete	Coffee Beans 1kg	1899.99	1900
☐		Edit		Copy		Delete	Green Tea Box	650.00	700
☐		Edit		Copy		Delete	Bluetooth Speaker	7500.00	7500

Console

B5 — Find Products with Odd Product IDs

Objective:

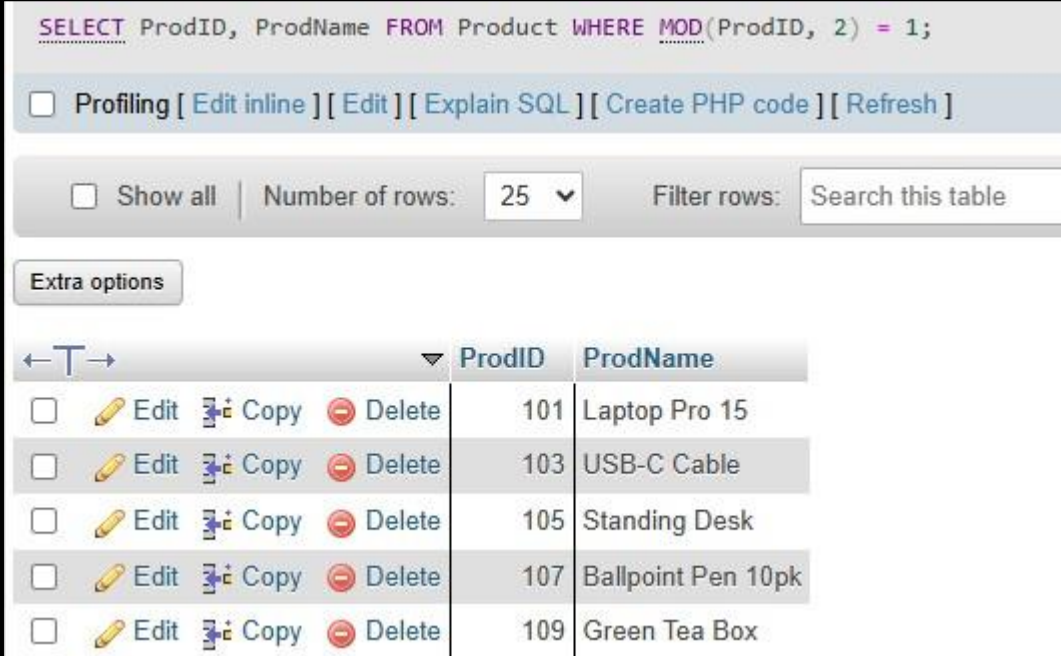
List all products whose ProdID is an odd number.

SQL Query:

```
SELECT
    ProdID,
    ProdName
FROM Product
WHERE MOD(ProdID, 2) = 1;
```

Function Used: MOD()

Explanation: MOD() returns the remainder after division. If the remainder after dividing ProdID by 2 is 1, the number is odd.



The screenshot shows a SQL query execution interface. At the top, the query is displayed: `SELECT ProdID, ProdName FROM Product WHERE MOD(ProdID, 2) = 1;`. Below the query, there are several interactive links: [Profiling](#), [Edit inline](#), [Edit](#), [Explain SQL](#), [Create PHP code](#), and [Refresh](#). A control bar includes a [Show all](#) checkbox, a "Number of rows:" dropdown set to 25, and a "Filter rows:" search box. An "Extra options" button is also present. The results are shown in a table with two columns: ProdID and ProdName. Each row includes a checkbox, an edit icon, a copy icon, and a delete icon.

	ProdID	ProdName
☐ Edit Copy Delete	101	Laptop Pro 15
☐ Edit Copy Delete	103	USB-C Cable
☐ Edit Copy Delete	105	Standing Desk
☐ Edit Copy Delete	107	Ballpoint Pen 10pk
☐ Edit Copy Delete	109	Green Tea Box

B6 — Extract Year, Month Name and Day of Week

Objective:

Display the year, month name, and day of the week for each customer's joining date.

SQL Query:

```
SELECT
    CustID,
    CustName,
    YEAR(JoinDate) AS
    JoinYear,
    MONTHNAME(JoinDate)
    AS JoinMonth,
    DAYNAME(JoinDate) AS
    JoinDay
FROM Customer;
```

Functions Used: YEAR(), MONTHNAME(), DAYNAME()

Explanation:

YEAR() extracts the year.

MONTHNAME() returns the full month name.

DAYNAME() returns the name of the day.

`SELECT CustID, CustName, YEAR(JoinDate) AS JoinYear, MONTHNAME(JoinDate) AS JoinMonth, DAYNAME(JoinDate) AS JoinDay FROM Customer;`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				CustID	CustName	JoinYear	JoinMonth	JoinDay
☐	Edit	Copy	Delete	1	Ali Khan	2022	January	Saturday
☐	Edit	Copy	Delete	2	Sara Iqbal	2022	April	Friday
☐	Edit	Copy	Delete	3	HAMZA RAZA	2023	February	Friday
☐	Edit	Copy	Delete	4	Ayesha Noor	2023	May	Thursday
☐	Edit	Copy	Delete	5	bilal ahmed	2023	September	Friday
☐	Edit	Copy	Delete	6	Fatima Sheikh	2024	January	Friday
☐	Edit	Copy	Delete	7	Usman Tariq	2024	June	Sunday
☐	Edit	Copy	Delete	8	Maira Javed	2024	August	Sunday

B7—Format Date of Birth

Objective:

Display each customer's date of birth in the format DD-Month-YYYY.

SQL Query:

SELECT

CustName,

DOB,

DATE_FORMAT(DOB,

'%d-%M-%Y') AS FormattedDOB

FROM Customer;

Function Used: DATE_FORMAT()

Explanation: DATE_FORMAT() changes a date into a readable format.

For example:

1995-04-12 *becomes:*

12-April-1995

`SELECT CustName, DOB, DATE_FORMAT(DOB, '%d-%M-%Y') AS FormattedDOB FROM Customer;`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: | Sort by key: None

Extra options

				CustName	DOB	FormattedDOB
☐				Ali Khan	1995-04-12	12-April-1995
☐				Sara Iqbal	1998-11-20	20-November-1998
☐				HAMZA RAZA	1997-08-05	05-August-1997
☐				Ayesha Noor	1999-02-14	14-February-1999
☐				bilal ahmed	2000-06-30	30-June-2000
☐				Fatima Sheikh	1996-10-25	25-October-1996
☐				Usman Tariq	2001-03-18	18-March-2001
☐				Maira Javed	1994-12-09	09-December-1994

B8—Calculate Current Age

Objective:

Calculate the current age of every customer in years.

SQL Query:

```
SELECT
    CustName,
    DOB,
    TIMESTAMPDIFF(YEAR,
    DOB, CURDATE()) AS Age
FROM Customer;
```

Functions Used: TIMESTAMPDIFF(), CURDATE()

Explanation:

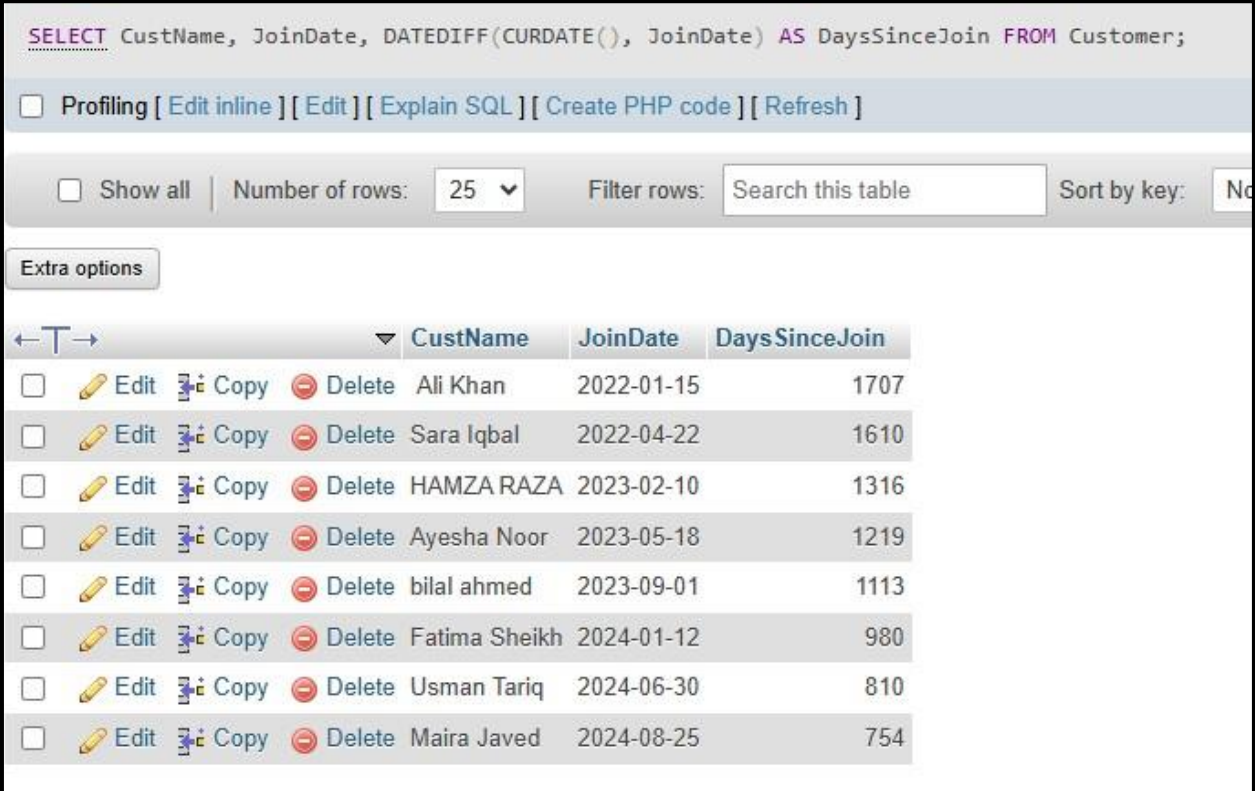
TIMESTAMPDIFF() calculates the difference between the date of birth and today's date in years.

CURDATE() returns the current date.

Explanation: DATEDIFF() calculates the difference between two dates in days.

The calculation is:

Today's Date - Join Date



The screenshot shows a SQL query execution interface. At the top, the query is: `SELECT CustName, JoinDate, DATEDIFF(CURDATE(), JoinDate) AS DaysSinceJoin FROM Customer;`. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. A control bar shows 'Show all', 'Number of rows: 25', 'Filter rows: Search this table', and 'Sort by key: No'. An 'Extra options' button is also present. The results are displayed in a table with columns: CustName, JoinDate, and DaysSinceJoin. Each row includes checkboxes for Edit, Copy, and Delete actions.

				CustName	JoinDate	DaysSinceJoin
☐	Edit	Copy	Delete	Ali Khan	2022-01-15	1707
☐	Edit	Copy	Delete	Sara Iqbal	2022-04-22	1610
☐	Edit	Copy	Delete	HAMZA RAZA	2023-02-10	1316
☐	Edit	Copy	Delete	Ayesha Noor	2023-05-18	1219
☐	Edit	Copy	Delete	bilal ahmed	2023-09-01	1113
☐	Edit	Copy	Delete	Fatima Sheikh	2024-01-12	980
☐	Edit	Copy	Delete	Usman Tariq	2024-06-30	810
☐	Edit	Copy	Delete	Maira Javed	2024-08-25	754

B10 — Customers Who Joined in 2023

Objective:

List customers who joined during the year 2023 using a date function.

SQL Query:

```
SELECT
    CustID,
    CustName,
    JoinDate
FROM Customer
```

```
WHERE YEAR(JoinDate) =
```

```
2023;
```

Function Used: YEAR()

Explanation: YEAR() extracts the year from JoinDate. The query then filters customers whose joining year is 2023.

The screenshot shows a SQL query execution interface. The query is: `SELECT CustID, CustName, JoinDate FROM Customer WHERE YEAR(JoinDate) = 2023;`. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. There are also controls for Show all, Number of rows (set to 25), Filter rows (Search this table), and Sort by key (None). An Extra options button is also present. The results are displayed in a table with columns CustID, CustName, and JoinDate. The table contains three rows of data.

	CustID	CustName	JoinDate
☐ Edit Copy Delete	3	HAMZA RAZA	2023-02-10
☐ Edit Copy Delete	4	Ayesha Noor	2023-05-18
☐ Edit Copy Delete	5	bilal ahmed	2023-09-01

B11 — Products Launched in Q4

Objective:

List products launched during October, November, or December of any year.

SQL Query:

```
SELECT
    ProdID,
    ProdName,
    LaunchDate
FROM Product
WHERE
    MONTH(LaunchDate) IN (10,
    11, 12);
```

Function Used: MONTH()

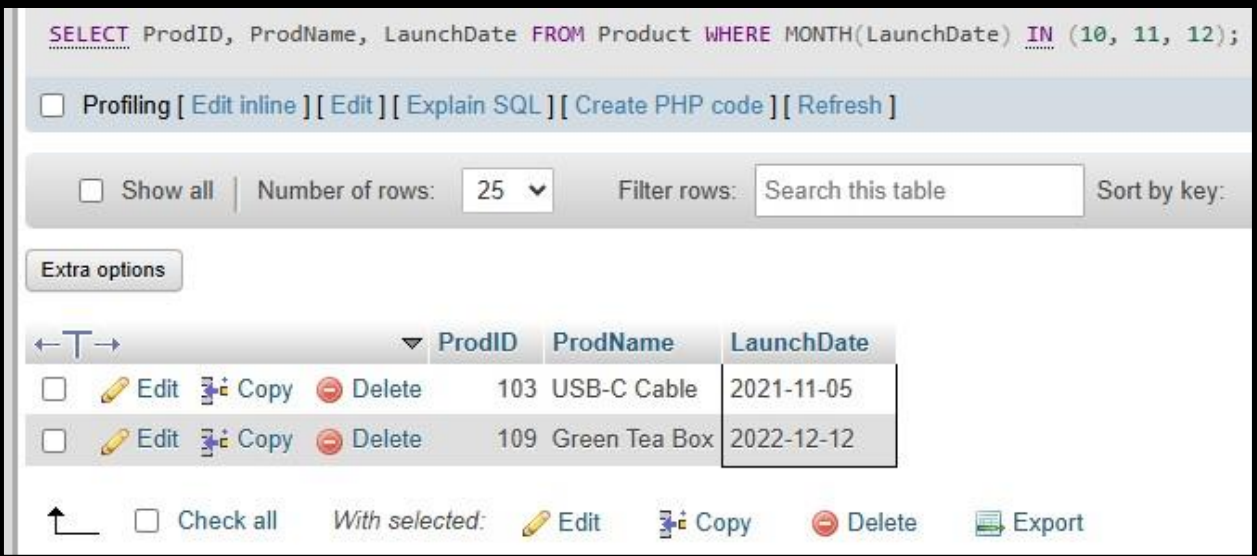
Explanation: Q4 consists of three months:

October = 10

November = 11

December = 12

The IN operator checks whether the month belongs to these three values.



The screenshot shows a SQL query execution interface. The query is: `SELECT ProdID, ProdName, LaunchDate FROM Product WHERE MONTH(LaunchDate) IN (10, 11, 12);`. Below the query, there are options for Profiling, Edit inline, Edit, Explain SQL, Create PHP code, and Refresh. There are also controls for Show all, Number of rows (set to 25), Filter rows (Search this table), and Sort by key. An Extra options button is also present. The results table has three columns: ProdID, ProdName, and LaunchDate. It contains two rows: one for ProdID 103 (USB-C Cable) with LaunchDate 2021-11-05, and another for ProdID 109 (Green Tea Box) with LaunchDate 2022-12-12. At the bottom, there are checkboxes for Check all, and buttons for Edit, Copy, Delete, and Export.

	ProdID	ProdName	LaunchDate
☐	103	USB-C Cable	2021-11-05
☐	109	Green Tea Box	2022-12-12

B12 — Customers Who Joined in the Last 6 Months

Objective:

Find customers who joined within the last six months from today's date.

SQL Query:

SELECT

CustID,

CustName,

JoinDate

FROM Customer

WHERE JoinDate >=

DATE_SUB(CURDATE(),

INTERVAL 6 MONTH);

Functions Used: DATE_SUB(), CURDATE()

Explanation: DATE_SUB() subtracts six months from today's date. Customers whose JoinDate is on or after that date are selected.

```
SELECT CustID, CustName, JoinDate FROM Customer WHERE JoinDate >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH);
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

B13—Product Age in Days

Objective:

Calculate how many days have passed since each product was launched.

SQL Query:

SELECT

ProdName,

LaunchDate,

DATEDIFF(CURDATE(),

LaunchDate) AS AgeInDays

FROM Product;

Functions Used: DATEDIFF(), CURDATE()

Explanation: The query calculates the number of days between today's date and the product's launch date.

`SELECT ProdName, LaunchDate, DATEDIFF(CURDATE(), LaunchDate) AS AgeInDays FROM Product;`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: | Sort by key: N

Extra options

				ProdName	LaunchDate	AgeInDays
☐				Laptop Pro 15	2023-03-10	1288
☐				Wireless Mouse	2022-07-22	1519
☐				USB-C Cable	2021-11-05	1778
☐				Office Chair	2023-01-15	1342
☐				Standing Desk	2024-02-28	933
☐				Notebook A4	2020-04-01	2361
☐				Ballpoint Pen 10pk	2020-04-01	2361
☐				Coffee Beans 1kg	2023-09-20	1094
☐				Green Tea Box	2022-12-12	1376
☐				Bluetooth Speaker	2024-05-18	853

B14 — Date 90 Days After Launch

Objective:

Calculate the date exactly 90 days after each product's launch date.

SQL Query:

```
SELECT
    ProdName,
    LaunchDate,
    DATE_ADD(LaunchDate,
    INTERVAL 90 DAY) AS
    NinetyDaysLater
FROM Product;
```

Function Used: DATE_ADD()

Explanation: DATE_ADD() adds a specified amount of time to a date. Here, 90 days are added to each product's LaunchDate.

`SELECT ProdName, LaunchDate, DATE_ADD(LaunchDate, INTERVAL 90 DAY) AS NinetyDaysLater FROM Product;`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

		ProdName	LaunchDate	NinetyDaysLater
☐	Edit	Copy	Delete	Laptop Pro 15
				2023-03-10
				2023-06-08
☐	Edit	Copy	Delete	Wireless Mouse
				2022-07-22
				2022-10-20
☐	Edit	Copy	Delete	USB-C Cable
				2021-11-05
				2022-02-03
☐	Edit	Copy	Delete	Office Chair
				2023-01-15
				2023-04-15
☐	Edit	Copy	Delete	Standing Desk
				2024-02-28
				2024-05-28
☐	Edit	Copy	Delete	Notebook A4
				2020-04-01
				2020-06-30
☐	Edit	Copy	Delete	Ballpoint Pen 10pk
				2020-04-01
				2020-06-30
☐	Edit	Copy	Delete	Coffee Beans 1kg
				2023-09-20
				2023-12-19
☐	Edit	Copy	Delete	Green Tea Box
				2022-12-12
				2023-03-12
☐	Edit	Copy	Delete	Bluetooth Speaker
				2024-05-18
				2024-08-16

Console

B15 — Combined Customer Summary

Objective:

Create a single summary column containing the customer's uppercase name, age, and joining month/year.

SQL Query:

```
SELECT
  CONCAT(
    'Hello ',
    UPPER(TRIM(CustName)),
    ', age ',
    TIMESTAMPDIFF(YEAR,
      DOB, CURDATE()),
    ', joined ',
```

```
DATE_FORMAT(JoinDate,  
'%b %Y')  
) AS Summary
```

```
FROM Customer;
```

Required Format: Hello ALI KHAN, age 31, joined Jan 2022

Functions Used:

- CONCAT()
 - UPPER()
 - TRIM()
 - TIMESTAMPDIFF()
 - CURDATE()
 - DATE_FORMAT()
- Example Output:**

Hello ALI KHAN, age 31, joined Jan 2022

Showing rows 0 - 7 (8 total, Query took 0.0006 seconds.)

```
SELECT CONCAT( 'Hello ', UPPER(TRIM(CustName)), ', age ', TIMESTAMPDIFF(YEAR, DOB, CURDATE()), ', joined ', DATE_FORMAT(JoinDate, '%b %Y') ) AS  
Summary FROM Customer;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

Summary

Hello ALI KHAN, age 31, joined Jan 2022
Hello SARA IQBAL, age 27, joined Apr 2022
Hello HAMZA RAZA, age 29, joined Feb 2023
Hello AYESHA NOOR, age 27, joined May 2023
Hello BILAL AHMED, age 26, joined Sep 2023
Hello FATIMA SHEIKH, age 29, joined Jan 2024
Hello USMAN TARIQ, age 25, joined Jun 2024
Hello MAIRA JAVED, age 31, joined Aug 2024

Activate Windows

Conclusion

In Part B, numeric and date/time scalar functions were used to perform calculations and transform database values into meaningful information. The tasks demonstrated discounts, sales tax, rounding, odd-number filtering, date extraction, date formatting, age calculation, date differences, date filtering, and date arithmetic.

*****THE END*****